Swinburne University of Technology

Faculty of Science, Engineering and Technology

ASSIGNMENT COVER SHEET

Subject Code: COS30008

Subject Title: Data Structures and Patterns

Student ID: 104198996

StudentName: Lau Ngoc Quyen

Assignment number and title: 1, Solution Design in C++

Due date: 7th October, 2023

Lecturer: Dr. Ky Trung Pham

Check Tutorial	Mon 10:30	Mon 14:30	Tues 08:30	Tues 10:30	Tues 12:30	Tues 14:30	Tues 16:30	Wed 08:30	Wed 10:30	Wed 12:30	Wed 14:30
					X						

Marker's comments:

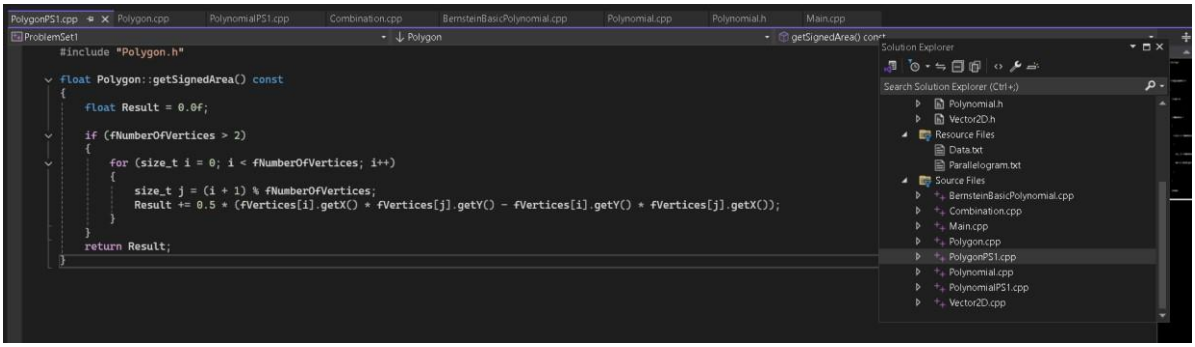
Problem	Marks	Obtained
1	38	
2	60	
3	38	
4	20	
Total	156	

Extension certification:

This assignment has been given an extension and is now due on 7th October

Signature of Convener:_____

PolygonPS1.cpp:



PolymonialPS1.cpp:

ProblemSet1 Polynomial

```cpp
#include "Polynomial.h"
#include <cmath>

using namespace std;

double Polynomial::operator()(double aX) const {
    double result = 0.0;

    for (size_t i = 0; i <= fDegree; i++) {
        result += fCoeffs[i] * std::pow(aX, i);
    }
    return result;
}

Polynomial Polynomial::getDerivative() const {
    Polynomial result;

    if (fDegree == 0) {
        return result;
    }

    result.fDegree = fDegree - 1;

    for (size_t i = 1; i <= fDegree; i++) {
        result.fCoeffs[i - 1] = fCoeffs[i] * i;
    }

    return result;
}

Polynomial Polynomial::getIndefiniteIntegral() const {
    Polynomial result;

    result.fDegree = fDegree + 1;

    for (size_t i = 0; i <= fDegree; i++) {
        result.fCoeffs[i + 1] = fCoeffs[i] / (i + 1);
    }

    return result;
}

double Polynomial::getDefiniteIntegral(double aXLow, double aXHigh) const {
    return this->getIndefiniteIntegral()(aXHigh) - this->getIndefiniteIntegral()(aXLow);
}
```

Combination.cpp:

```
ProblemSet1                                                    Combination
#include "Combination.h"

Combination::Combination(size_t aN, size_t aK) : fN(aN), fK(aK)
{}

size_t Combination::getN() const
{
    return fN;
}

size_t Combination::getK() const
{
    return fK;
}

unsigned long long Combination::operator()() const
{
    if (fK > fN) return 0ll;
    unsigned long long Result = 1;

    for (size_t i = 0; i < fK; i++) {
        Result *= (fN - i);
        Result /= (i + 1);
    }

    return Result;
}
```

BernsteinBasicPolynomial.cpp:

```
PolygonPS1.cpp  Polygon.cpp  PolynomialPS1.cpp  Combination.cpp  BernsteinBasicPolynomial.cpp  Polynomial.cpp  Polynomial.h  Main.cpp
ProblemSet1                                  BernsteinBasisPolynomial                    operator()(double aX) const

#include "BernsteinBasisPolynomial.h"
#include <cmath>

BernsteinBasisPolynomial::BernsteinBasisPolynomial(unsigned int aV, unsigned int aN) : fFactor(Combination(aN, aV))
{}

double BernsteinBasisPolynomial::operator()(double aX) const
{
    return fFactor() * pow(aX, fFactor.getK()) * pow((1 - aX), (fFactor.getN() - fFactor.getK()));
}
```

```
Solution Explorer
Search Solution Explorer (Ctrl+;)
  Polynomial.h
  Vector2D.h
  Resource Files
    Data.txt
    Parallelogram.txt
  Source Files
    BernsteinBasicPolynomial.cpp
    Combination.cpp
    Main.cpp
    Polygon.cpp
    PolygonPS1.cpp
    Polynomial.cpp
    PolynomialPS1.cpp
    Vector2D.cpp
```